Eventpup

Eventpup is a full-stack event platform built with Next.js 14 and TypeScript. Visitors can browse and search events, signed-in users can create and manage their own events, and attendees can book tickets through Stripe checkout.

This was a personal project I built to practice an end-to-end Next.js application — authentication, a real database, form validation, image uploads, payments, and webhooks all working together — rather than a static UI only.

Features

- **Browse and search events** — events are shown as cards on the home page, with a search bar and an event detail page for each event.
- **Create, edit, and delete events** — organizers use a validated event form covering title, description, location, image, start and end date/time, category, price (or free), and an external URL.
- **Image uploads** — event images are uploaded through UploadThing using a drag-and-drop file uploader.
- **Authentication with Clerk** — sign-in and sign-up flows, with protected routes handled by Clerk middleware. Clerk webhooks keep the MongoDB user records in sync when a user is created, updated, or deleted.
- **Ticket checkout with Stripe** — paid events go through Stripe checkout, and a Stripe webhook creates the order record once the checkout session completes.
- **Profile and orders pages** — users can view the events they organized, the tickets they ordered, and order details.

Tech Stack

- **Framework:** Next.js 14 (App Router, Server Actions), React 18
- **Language:** TypeScript
- **Styling:** Tailwind CSS, Radix UI components
- **Forms & validation:** React Hook Form, Zod
- **Authentication:** Clerk (with Svix webhook verification)
- **Database:** MongoDB, accessed with Mongoose (User, Event, Category, and Order models)
- **Payments:** Stripe (Checkout + webhooks)
- **File uploads:** UploadThing

Implementation Highlights

- Server Actions in `lib/actions` handle event, order, user, and category operations, instead of separate API routes for CRUD.
- The event form is validated by a single Zod schema (`lib/validator.ts`), so field rules are defined once and reused.
- The Mongoose connection is cached (`lib/database/index.ts`), so the app reuses the existing connection instead of opening a new one on every request.
- Webhooks are treated as the source of truth for two sync jobs: Clerk keeps user records up to date, and Stripe creates an order only after a payment actually completes.

Getting Started

Prerequisites

- Node.js 18 or later
- A MongoDB database (for example, a free MongoDB Atlas cluster)
- Accounts for Clerk, Stripe, and UploadThing (free tiers are enough to run it locally)

Installation

```
git clone https://github.com/trieuvi2032/Eventpup.git
cd Eventpup
npm install
```

Create a `.env.local` file in the project root with your own keys:

```
# Clerk
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=
CLERK_SECRET_KEY=
NEXT_PUBLIC_CLERK_SIGN_IN_URL=/sign-in
NEXT_PUBLIC_CLERK_SIGN_UP_URL=/sign-up
NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL=/
NEXT_PUBLIC_CLERK_AFTER_SIGN_UP_URL=/
WEBHOOK_SECRET=

# MongoDB
MONGODB_URI=

# UploadThing
UPLOADTHING_SECRET=
UPLOADTHING_APP_ID=

# Stripe
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
```

```
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=

# App
NEXT_PUBLIC_SERVER_URL=http://localhost:3000
```

Then run the development server:

```
npm run dev
```

Open http://localhost:3000 in your browser.

> Never commit your real `.env.local` file or any live keys to the repository. The list above is a template only.

Project Structure

```
app/
  (auth)/          Sign-in and sign-up pages (Clerk)
  (root)/          Home page, events, orders, and profile pages
  api/
    uploadthing/   Upload endpoint
    webhook/       Clerk and Stripe webhook handlers
components/
  shared/          Event cards, event form, checkout, search, header, footer
  ui/              Reusable UI primitives (Radix-based)
lib/
  actions/         Server Actions for events, orders, users, categories
  database/        Mongoose connection and models
  uploadthing.ts   Upload configuration
  validator.ts     Zod schema for the event form
types/             Shared TypeScript types
middleware.ts      Clerk route protection
```

Status

Personal learning project. It is fully functional end to end (auth, database, payments, uploads), but it is not a production deployment and uses test-mode Stripe keys.